



NEURICK Educational Kit Manual



Ultimate guide for the **NEURICK**
AI & Robotics development **board**



EXPLODED VIEW

How the Neurick board is organised

The Neurick main board carries everything a robotics project needs on a single PCB: four DC motor channels with quadrature encoder inputs, three servo ports, a 0.91-inch OLED status display, a six-axis motion sensor, an enclosed microphone and speaker, a microSD card slot, a 40-pin expansion header and the battery and charging circuitry. What makes it different from a plain development board is that it runs on **two microcontrollers** instead of one, each with a clearly defined job.

Controller	Responsibility
ESP32-S3	The application controller — this is the one you program. It runs your sketch, owns the expansion header, the OLED display, the MPU6050 motion sensor, the microphone, the speaker, the microSD card and the Wi-Fi and Bluetooth radios.
STM32F103	The motion controller — factory programmed, not user programmable. It drives the four DC motor channels, counts the four quadrature encoders, measures motor current and battery voltage, reads the onboard user button and generates the pulses for the three servo channels.

The two controllers talk to each other over an I2C link. You never write code for the STM32 and you never need to flash it — it ships with its motion firmware already installed. Instead, you include the **Newrick library** in your ESP32 sketch and call its functions. Each call is packed into an I2C message, sent across to the STM32, and acted upon within milliseconds.

The short version. Everything you write goes to the ESP32-S3 over USB. Anything that moves — motors and servos — is reached by calling the Newrick library, which forwards the request to the STM32 on your behalf.

Why the work is split this way

- **Motion never stutters.** Counting four quadrature encoders means servicing tens of thousands of edge interrupts per second. Handing that to a dedicated controller means a slow web request or an SD card write in your sketch can never cause a missed encoder count or a glitched servo pulse.
- **Your sketch cannot damage the hardware.** Direction and PWM signals come from firmware that will not switch both sides of an H-bridge on at once, so a mistake cannot short a motor driver.
- **Your code stays short.** Driving a motor is one line — `nr.motor(1, 600);` — instead of a page of pin definitions, PWM channel allocations and direction logic.
- **More pins are left for you.** Because motor, encoder and servo signals never reach the ESP32, twenty of its GPIOs are free and brought out to the expansion header.

What you can and cannot change

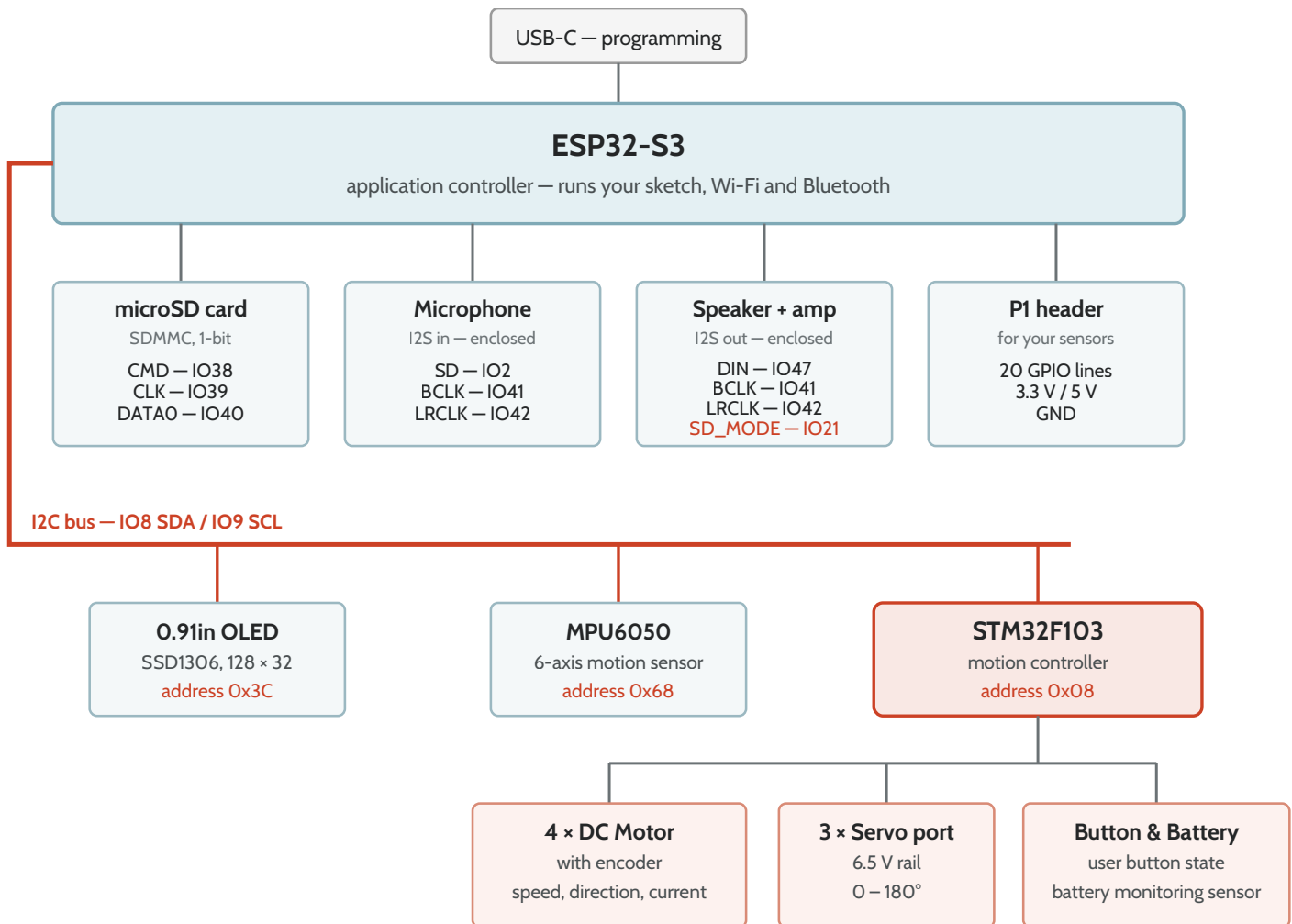
Task	How it is done
Writing and uploading firmware	Arduino IDE over the USB-C port, directly to the ESP32-S3.
Driving motors and servos	Newrick library function calls from your ESP32 sketch.
Reading encoders, current, battery and the user button	Newrick library variables, refreshed by an update call.
Display, motion sensor, audio, SD card, external sensors	Ordinary Arduino code on the ESP32 — no library needed.
STM32 motion firmware	Factory programmed. Not user accessible and not required for any laboratory in this manual.

EXPLODED VIEW



Internal architecture

This diagram shows how the parts of the Neurick board are wired to one another inside the enclosure. Everything below the red I2C line is reached over that one shared bus; everything hanging directly off the ESP32-S3 has its own dedicated pins, listed in each block.



Internal architecture of the Neurick board. Pin numbers are ESP32-S3 GPIOs.

Reading the diagram

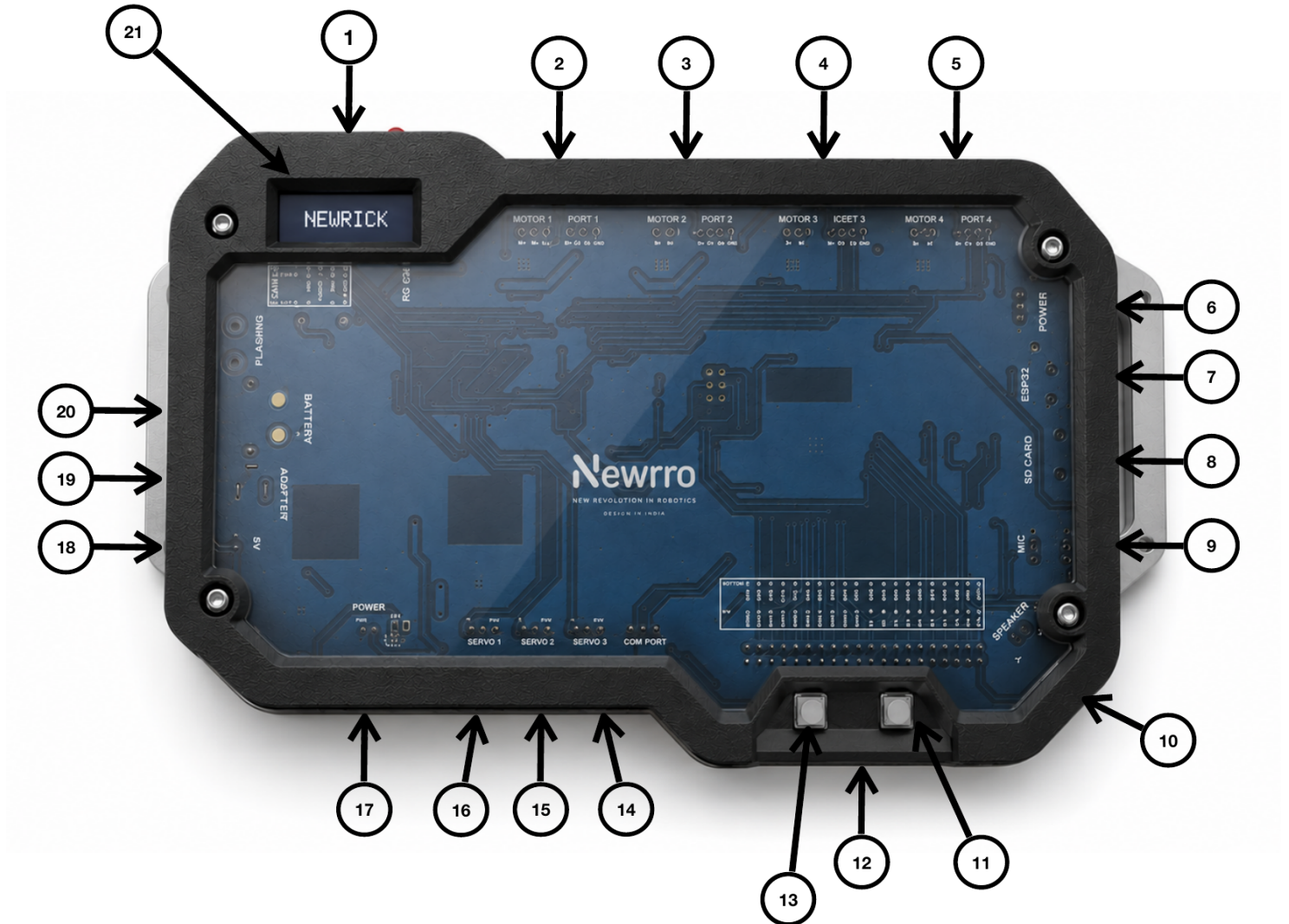
- The **ESP32-S3** is the only part you program, and the USB-C port goes straight to it.
- The **microSD card**, **microphone**, **speaker** and **P1 header** sit on dedicated ESP32 pins. The mic and speaker are enclosed — their pins are shown so you can use them in a program, not wire them.
- The **mic** and **speaker** share **BCLK** and **LRCLK**: two devices on one audio bus, which is why a program listens or plays but never both at once. The amplifier's **SD_MODE** on **IO21** is held low by a resistor, so it stays silent until your program drives that pin high.
- The **I2C bus** carries the display, the motion sensor and the STM32, so one `nr.begin()` starts the bus for everything.
- The **STM32F103** is just another device on that bus, at address **0x08**. Motor and servo signals never reach the ESP32 — you send a request and the STM32 acts on it.
- The **user button** and **battery monitor** are wired to the **STM32** too. That is why you read them with `nr.buttonState` and `nr.batteryVolts` after an update call rather than with `digitalRead()` — and why they cost you no GPIO pin.



EXPLODED VIEW

Neurick board layout

Every callout on the diagram below maps to a physical connector, module or terminal on the Neurick board. Use it together with the reference table on the following page when you are wiring a new sensor or actuator.



The Neurick main board, with every connector, module and interface called out by number.

EXPLODED VIEW



Interface and connector reference

Read this table alongside the board diagram. The **Owner** column tells you which controller the connector belongs to, and therefore how you reach it from your program: interfaces owned by the STM32 are reached through the Newrick library, while interfaces owned by the ESP32 are reached with ordinary Arduino code.

Ref	Component	Owner	Description
1	STM32 programmer	STM32	Programming header for the motion controller. Used at the factory — the firmware is already installed and you never need this.
2	Motor 1 with encoder	STM32	Screw terminals for DC motor 1 and the matching quadrature encoder connector.
3	Motor 2 with encoder	STM32	Screw terminals for DC motor 2 and its encoder connector.
4	Motor 3 with encoder	STM32	Screw terminals for DC motor 3 and its encoder connector.
5	Motor 4 with encoder	STM32	Screw terminals for DC motor 4 and its encoder connector.
6	5 V USB power	—	USB input used to power the board.
7	USB-C for ESP programming	ESP32	Programming and Serial Monitor connection. Every sketch in this manual is uploaded here.
8	SD card slot	ESP32	microSD slot on the ESP32 SDMMC interface, 1-bit mode.
9	Mic	ESP32	I2S digital microphone, enclosed and wired internally. Nothing to connect.
10	Speaker	ESP32	Speaker and amplifier, enclosed and wired internally. Nothing to connect.
11	User button	STM32	General-purpose push button, read by the motion controller and reported to your sketch through the library.
12	GPIO output slot	ESP32	The P1 expansion header — twenty ESP32 GPIO lines plus 3.3 V, 5 V and ground.
13	MCU reset button	—	Restarts the board. Your program runs again from the top.
14	Servo 3	STM32	Three-pin servo header — signal, 6.5 V and ground. Servo channel 3.
15	Servo 2	STM32	Three-pin servo header. Servo channel 2.
16	Servo 1	STM32	Three-pin servo header. Servo channel 1.
17	6.5 V power	—	Servo supply rail.
18	5 V power	—	Regulated 5 V supply point for external modules.
19	Charging slot	—	Charging input for the onboard battery.
20	Battery slot	—	Battery input terminals for the 12 V pack, marked – and +.
21	Display	ESP32	0.91-inch SSD1306 OLED, 128 × 32, on the I2C bus at address 0x3C.

Three parts of the board carry no callout because they are surface-mounted chips rather than connectors: the **ESP32-S3**, the **STM32F103** and the **MPU6050** motion sensor. The internal architecture diagram earlier in this chapter shows how all three fit together.

EXPLODED VIEW



Power, charging and safe operating limits

The Neurick board runs from a rechargeable battery pack connected to the battery terminals, switched by the main power switch. The pack feeds the motor drivers directly and also feeds the onboard regulators that produce the 5 V and 3.3 V rails used by the logic, the sensors and the expansion header.

Powering the board

- **Battery.** Connect the supplied **12 V** pack to the battery terminals, observing the – and + markings. Reversing the pack can damage the board permanently.
- **Charging.** Use the charger slot with the supplied charger only. Charge with the main switch off, and do not leave a pack charging unattended.
- **USB.** The USB-C port powers the ESP32-S3 and is enough for programming, the display and small sensors. It is **not** enough to run motors — a motor command issued while the board is on USB power alone will either do nothing or brown out the board. Switch the battery on before driving anything.

What each rail is for

Rail	Purpose and limits
12 V battery rail	Feeds the four motor driver channels directly. The rail sags as the pack discharges, so motor speed falls with it — monitor it with <code>batteryVolts</code> .
6.5 V servo rail	Dedicated supply on the three servo headers. Sized for standard hobby servos; it is not a general-purpose output.
5 V	Available on the expansion header and the 5 V module header. Use it for sensor modules that expect 5 V logic supply, such as ultrasonic and gas sensors.
3.3 V	Available on the expansion header. This is the same logic level the ESP32 uses, so any sensor powered from it can be wired to a GPIO directly.

Crucial note: ESP32-S3 GPIO pins are **3.3 V logic and are not 5 V tolerant**. A sensor powered from the 5 V rail that returns a 5 V digital or analog signal must be brought down with a voltage divider or a level shifter before it touches a GPIO. Connecting a 5 V signal directly to a GPIO can destroy the pin.

Safe operating practice

- Switch the board off before changing any wiring on the expansion header or the motor terminals.
- Never connect a motor or a pump to a GPIO pin. Inductive loads belong on the motor channels, which have the required driver and protection circuitry.
- Keep the total draw of everything you hang on the 5 V and 3.3 V header pins modest. These rails are sized for sensor modules, not for extra actuators.
- Do not short the 5 V or 3.3 V pins to ground, and double-check polarity before switching on — the header has no reverse-polarity protection.
- Stop the motors before the pack is fully flat. Running a pack down hard shortens its life, and a sagging rail resets the ESP32 mid-program.



EXPANSION HEADER

The P1 40-pin header

P1 is a 2.54 mm, 2 × 20 header on the edge of the board. It carries **twenty ESP32-S3 GPIO lines** together with 3.3 V, 5 V and ground rails, and it is how every external sensor and module in this manual is connected.

Pin numbering follows the usual convention for a dual-row header: **odd pins run down one column and even pins down the other**, with pin 1 at the marked end. The odd column is entirely power and ground; the even column is entirely signal.

Pin	Function	Pin	Signal	Notes
1	3.3 V	2	GPIO4	ADC1 — safe for analog input with Wi-Fi on.
3	3.3 V	4	GPIO5	ADC1 — safe for analog input with Wi-Fi on.
5	5 V	6	GPIO6	ADC1 — safe for analog input with Wi-Fi on.
7	5 V	8	GPIO7	ADC1 — safe for analog input with Wi-Fi on.
9	5 V	10	GPIO15	Digital. ADC2 — analog unusable while Wi-Fi is on.
11	5 V	12	GPIO16	Digital. ADC2 — analog unusable while Wi-Fi is on.
13	5 V	14	GPIO17	Digital. ADC2 — analog unusable while Wi-Fi is on.
15	5 V	16	GPIO18	Digital. ADC2 — analog unusable while Wi-Fi is on.
17	5 V	18	GPIO8 — SDA	Shared I2C data line. See the I2C section before using.
19	5 V	20	GPIO3	ADC1. Strapping pin — see cautions below.
21	GND	22	GPIO46	Strapping pin — prefer as an input; see cautions below.
23	GND	24	GPIO9 — SCL	Shared I2C clock line. See the I2C section before using.
25	GND	26	GPIO10	ADC1 — safe for analog input with Wi-Fi on.
27	GND	28	GPIO11	Digital. ADC2 — analog unusable while Wi-Fi is on.
29	GND	30	GPIO12	Digital. ADC2 — analog unusable while Wi-Fi is on.
31	GND	32	GPIO13	Digital. ADC2 — analog unusable while Wi-Fi is on.
33	GND	34	GPIO14	Digital. ADC2 — analog unusable while Wi-Fi is on.
35	GND	36	GPIO48	Digital, no analog input.
37	GND	38	GPIO45	Strapping pin — see cautions below.
39	GND	40	GPIO1	ADC1 — safe for analog input with Wi-Fi on.

Each laboratory in this manual names a specific pin so you can wire it and run the program straight away, but none of those choices is fixed. Any pin from this table will do, provided it suits the sensor — an ADC1 pin for anything analog, any free digital pin otherwise. Change the constant at the top of the program to match whatever you wired.



EXPANSION HEADER

Power rails on the header

Rail	Pins	Use
3.3 V	1, 3	Logic-level supply. Anything powered from here can be wired straight to a GPIO with no level shifting.
5 V	5 – 19 (odd)	For modules that need 5 V to work properly, such as ultrasonic and gas sensors. Their outputs may need a divider before reaching a GPIO.
GND	21 – 39 (odd)	Common ground. Every sensor must share this ground with the board or its readings will be meaningless.

There are eight 5 V pins and ten ground pins because a sensor-heavy project needs many of them — not because each pin carries a separate supply. All eight 5 V pins are the same rail, and all ten ground pins are the same node. Spreading your modules across different pins does not give you more current; it just saves you building a wiring harness.

Crucial note: the 3.3 V and 5 V header pins are fed by the onboard regulators, which are sized for sensor modules. Do not power motors, pumps, heaters, LED strips or a second development board from them. An actuator on a header pin will brown out the rail and reset the ESP32 mid-program, and a short will damage the regulator.

Analog inputs

The ESP32-S3 has two analog-to-digital converters, and the difference between them matters as soon as you switch on Wi-Fi.

- **ADC1 — GPIO1, 3, 4, 5, 6, 7, 8, 9 and 10.** Works at all times, including while Wi-Fi or Bluetooth is active. **Use these pins for every analog sensor.** On the header that means pins 40, 20, 2, 4, 6, 8 and 26 (leaving the two I2C lines free).
- **ADC2 — GPIO11 to GPIO18.** Shares hardware with the radio. Once Wi-Fi is started, `analogRead()` on these pins stops returning useful values. They are perfectly good digital pins — just do not read them as analog in a connected project.

`analogRead()` returns a value from **0 to 4095**, because the converter is 12-bit. 0 corresponds to 0 V and 4095 to roughly 3.3 V.

Pins committed to onboard hardware

Twenty of the ESP32-S3's pins reach the header. The rest are wired to the onboard peripherals and are not available to you. They are listed here so you know what each one does when it appears in a program, and so you are not surprised to find them missing from the header.

GPIO	Signal	Used by
GPIO0	Boot mode	Strapping pin, pulled up on the board.
GPIO2	SD_MIC	Microphone data.
GPIO19, GPIO20	USB D– / D+	The USB-C programming port.
GPIO21	SD_MODE	Speaker amplifier enable — HIGH wakes it, LOW shuts it down.
GPIO38	CMD_SD	microSD command line.
GPIO39	CLK_SD	microSD clock.



EXPANSION HEADER

GPIO	Signal	Used by
GPIO40	DO_SD	microSD data.
GPIO41	BCLK	Audio bit clock — shared by the microphone and the amplifier.
GPIO42	LRCLK	Audio word select — shared by the microphone and the amplifier.
GPIO43, GPIO44	TXDO / RXDO	Hardware serial port 0.
GPIO47	DIN_SPEAKER	Speaker amplifier data.

GPIO8 and GPIO9 carry the I2C bus and are the one exception: they serve the onboard display, motion sensor and motion controller *and* are brought out on header pins 18 and 24 so you can add your own I2C devices alongside them.

Pins that need care

Four ESP32-S3 pins are **strapping pins**: their voltage is sampled at the instant the chip comes out of reset to decide how it should start up. Three of them are brought out on P1.

Signal	Pin	What to watch for
GPIO3	20	Selects the JTAG signal source at boot. Normally free to use; leave it floating or lightly loaded during reset.
GPIO45	38	Selects the internal flash supply voltage at boot. Must not be pulled high while the board is starting.
GPIO46	22	Controls ROM boot messages. The most sensitive of the three — prefer it for a simple input and avoid driving it during reset.

The practical rule is simple: **wire your first sensors to the ordinary pins and keep the strapping pins for last**. If a board suddenly refuses to boot or to accept an upload after you attach something new, unplug whatever is on pins 20, 22 and 38 and try again — a sensor module holding one of these lines at the wrong level during power-up is the usual cause.

Two further notes. **GPIO48** (pin 36) is a plain digital pin with no analog input. And nothing on this header is **5 V tolerant** — a 5 V logic output must go through a divider or a level shifter before it reaches any of these pins.



EXPANSION HEADER

The shared I2C bus

The Neurick board has a single I2C bus, running on **GPIO8 (SDA)** and **GPIO9 (SCL)**. Three devices already live on it, and both lines are brought out to the header so you can add more.

Device	Address	Reached by
STM32F103 motion controller	0x08	The Newrick library.
SSD1306 OLED display	0x3C	The Adafruit SSD1306 library.
MPU6050 motion sensor	0x68	Direct register access with the Wire library.

I2C is a shared bus, so all four devices — the three above plus anything you add — use the same two wires and are told apart by address. That has two consequences worth understanding before you write any code.

Call `nr.begin()` once, and never call `Wire.begin()` again

`nr.begin()` starts the bus for the whole board: it calls `Wire.begin(8, 9)` and sets the clock to **400 kHz**. Once it has run, the display, the motion sensor and any I2C sensor you attach are all reachable — you simply skip the `Wire.begin()` line that a library's example sketch would normally contain.

Calling `Wire.begin()` a second time, or calling `Wire.setClock()` with a different speed, reconfigures the bus for every device on it. That is the single most common cause of a Neurick program in which the display works but the motors stop responding, or the other way round.

The rule. One `nr.begin()` in `setup()`, before anything else. No `Wire.begin()` anywhere. If a program in this manual does not use the motion controller at all, it may call `Wire.begin(8, 9)` itself — but never both.

Adding your own I2C sensor

Connect its SDA to pin 18, its SCL to pin 24, its supply to a 3.3 V pin and its ground to a ground pin. Most breakout boards already carry the pull-up resistors the bus needs, so nothing else is required. Check the new device's address against the table above — if it clashes with 0x08, 0x3C or 0x68 you must change it, usually with a solder jumper on the module.

Finding out what is on the bus

This short program lists the address of every device that answers. It is the fastest way to prove a new sensor is wired correctly, and a good first step whenever something has stopped working.

```
#include <Newrick.h>

Newrick nr;

void setup() {
  Serial.begin(115200);
  delay(2000);
  nr.begin();
  Serial.println("Scanning the I2C bus...");

  int found = 0;
  for (uint8_t addr = 1; addr < 127; addr++) {
    Wire.beginTransmission(addr);
    if (Wire.endTransmission() == 0) {
      Serial.print("  device at 0x");
      if (addr < 16) Serial.print("0");
      Serial.println(addr, HEX);
    }
  }
}
```



EXPANSION HEADER

```
        found++;
    }
}

Serial.print(found);
Serial.println(" device(s) found.");
}

void loop() { }
```

On a healthy board with nothing extra attached you should see exactly three devices: **0x08**, **0x3C** and **0x68**. A missing 0x08 means the motion controller is not powered — switch the battery on. A missing 0x3C or 0x68 usually means the bus has been reconfigured by a stray `Wire.begin()`.



NEWRICK LIBRARY

What the library does

The Newrick library is the bridge between the sketch you write and the motion controller that does the driving. It is a small library — six functions and ten variables — and once you know them you know everything the motion side of the board can do.

Every function works the same way underneath. Your call is packed into a short I2C message addressed to **0x08** and sent to the STM32, which acts on it and, where a reply is expected, sends data back. None of this is visible in your program: you write `nr.motor(1, 600)` and a motor turns.

The two halves of the library

- **Commands** — `motor()`, `servo()` and `resetEncoders()` send an instruction and return immediately. They are fire-and-forget.
- **Updates** — `updateEncoders()` and `updateSensors()` fetch a block of data from the motion controller and copy it into the library's public variables. The variables hold whatever the last successful update produced; they do not refresh by themselves.

That second point is the one beginners trip over. `nr.enc1` is not a live reading — it is a stored number. If you print it in a loop without calling `nr.updateEncoders()`, you will print the same value forever.

Getting started

```
#include <Newrick.h>

Newrick nr;    // one object for the whole sketch

void setup() {
  Serial.begin(115200);
  nr.begin();  // must run before any other Newrick call
}
```

Function reference

Function	What it does
<code>void begin(int sda_pin = 8, int scl_pin = 9)</code>	Starts the I2C bus at 400 kHz and clears all stored readings to zero. Call once in <code>setup()</code> , before anything else. The default pins are the ones the board is wired for — do not change them.
<code>void motor(uint8_t motor_id, int16_t speed)</code>	Sets the speed and direction of one motor. <code>motor_id</code> is 1 – 4; anything else is ignored. <code>speed</code> runs from –1000 to +1000 and is clamped to that range. 0 stops the motor.
<code>void servo(uint8_t s1_angle, uint8_t s2_angle, uint8_t s3_angle)</code>	Sets all three servo channels in a single command. Each angle is 0 – 180 degrees and is clamped to that range. Because one call sets every channel, you must pass all three angles every time.
<code>void resetEncoders()</code>	Tells the motion controller to zero all four encoder counts, and clears the local copies immediately. Use it to define a new starting point for a measurement.
<code>bool updateEncoders()</code>	Fetches all four encoder counts and stores them in <code>enc1</code> – <code>enc4</code> . Returns true on success, false if the motion controller did not answer or the data was corrupted. On failure the stored values are left untouched.



Function	What it does
<code>bool updateSensors()</code>	Fetches the four motor currents, the battery voltage and the button state, and stores them in the matching variables. Returns true on success, false on a failed or corrupted transfer.

Public variables

These are the values the two update functions fill in. Read them directly — `nr.enc1`, `nr.batteryVolts` and so on.

Variable	Type	Unit	Meaning
<code>enc1 ... enc4</code>	<code>int32_t</code>	counts	Cumulative encoder count for motors 1 – 4. Signed: counts up when the motor turns one way and down when it turns the other, so the value tells you net displacement, not distance travelled. Refreshed by <code>updateEncoders()</code> .
<code>current1 ... current4</code>	<code>float</code>	amperes	Current drawn by each motor channel. Refreshed by <code>updateSensors()</code> .
<code>batteryVolts</code>	<code>float</code>	volts	Battery pack voltage, measured through the onboard divider. Refreshed by <code>updateSensors()</code> .
<code>buttonState</code>	<code>uint8_t</code>	—	State of the onboard user button, reported as a byte. Zero when nothing is pressed. Refreshed by <code>updateSensors()</code> .

The motion controller measures raw voltages; the library converts them for you. Battery voltage is scaled back up by the ratio of the onboard divider, and each motor current is divided by the current-sense gain of **0.6 volts per ampere**. Both conversions are linear, so a reading that looks proportionally wrong — half of what you expect, or double — points to a wiring or supply problem rather than a software one.

How to read the values correctly

Always check the return value

I2C is a physical bus and transfers can fail — a loose jumper, a long lead or heavy electrical noise from a motor can all corrupt a message. The library guards against this with a checksum on every reply and reports the outcome to you.

```
if (nr.updateEncoders()) {
    Serial.println(nr.enc1);          // trust the value
} else {
    Serial.println("read failed");    // do NOT trust the value
}
```

Ignoring the return value is the classic Neurick bug. A failed read leaves the previous numbers in place, so a program that never checks appears to work — until the robot drives on a stale encoder count and overshoots.

Update once, use many times

Each update call is a complete I2C transaction. Calling `updateEncoders()` four times to read four encoders is four times the bus traffic for no benefit. Call it once, then use all four values.



```
// Correct
if (nr.updateEncoders()) {
    long total = nr.enc1 + nr.enc2 + nr.enc3 + nr.enc4;
}
```

Choose a sensible update rate

The bus is shared with the display and the motion sensor, so polling as fast as the processor allows will starve everything else. As a guide, refresh encoders up to about **50 times a second** for motion control, and the sensor block about **10 times a second** — battery voltage and button state simply do not change faster than that.

Commands persist until you change them

A motor keeps running at the last speed you set, and a servo holds the last angle, until a new command arrives or the board is powered down. There is no timeout. If your program ends, crashes or sits in a long blocking call, the motors keep going — so always stop them deliberately.

```
nr.motor(1, 0);
nr.motor(2, 0);
nr.motor(3, 0);
nr.motor(4, 0);
```

Quick reference

```
nr.begin();           // once, in setup()

nr.motor(1, 600);      // motor 1 forward at 60%
nr.motor(1, -600);     // motor 1 reverse at 60%
nr.motor(1, 0);        // motor 1 stop

nr.servo(90, 90, 90); // all three servos to centre

nr.resetEncoders();    // zero all four counts

if (nr.updateEncoders()) {
    // enc1, enc2, enc3 and enc4 now hold fresh counts
}

if (nr.updateSensors()) {
    // current1 to current4, batteryVolts and buttonState now hold fresh values
}
```

Remember. Motors need battery power — USB alone will not turn them. Speed is -1000 to +1000, not 0 to 255. Servo angles are 0 to 180 and all three are set together. And encoder and sensor values only change when you ask for them.



ENVIRONMENT SETUP

Setting up the Arduino IDE

Before we can make your board move, see or speak, we need to teach your computer how to talk to it. Everything in this manual is written for the Arduino IDE and is uploaded to the **ESP32-S3** over the USB-C port.

1. Download and install the Arduino IDE

- Go to the [Arduino Software](#) page and download the latest version of the Arduino IDE for your operating system (Windows, macOS or Linux).
- Run the installer and follow the on-screen instructions.

By default, the Arduino IDE does not know what an ESP32 is. Let's add it.

2. Install the ESP32 core

- Open the Arduino IDE.
- Go to **Tools > Board > Boards Manager...**
- In the search bar on the left, type **esp32**.
- Look for the package by **Espressif Systems** and click **Install**.
- Once installed, select your board from **Tools > Board > esp32**.

3. Configure the board

The ESP32 module on the Neurick board is selected as **ESP32S3 Dev Module**. Once the board is selected, set the following options in the **Tools** menu:

```
Board      : ESP32S3 Dev Module
Flash Size  : 16MB (128Mb)
PSRAM      : OPI PSRAM
USB CDC On Boot: Enabled
```

These settings matter. The Neurick module carries 16 MB of flash and octal-SPI PSRAM, and the audio laboratories will not run without PSRAM enabled. **USB CDC On Boot** lets the Serial Monitor open over the same USB cable you use to upload, which every laboratory in this curriculum relies on.

Before you upload. Select the correct serial port under **Tools > Port**, and set the Serial Monitor to **115200 baud** — every program in this curriculum initialises the serial link at that speed.

4. Install the Newrick library

The Newrick library is what lets your sketch reach the motion controller. Without it you can still use the display, the sensors and the expansion header, but you cannot drive a motor or a servo.

- Locate the **Newrick** library folder supplied with your kit. It contains two files, **Newrick.h** and **Newrick.cpp**.
- If you were given a ZIP file, go to **Sketch > Include Library > Add .ZIP Library...** and select it.
- If you were given a plain folder, copy the whole folder into your Arduino **libraries** directory — typically *Documents/Arduino/libraries* — so the path ends in *libraries/Newrick/Newrick.h*.
- Restart the Arduino IDE. The library is installed correctly when **Sketch > Include Library** lists **Newrick**.



ENVIRONMENT SETUP

Uploading your first program

Aim

To confirm that the ESP32-S3 is programmable, that the Serial Monitor is working, and that the Newrick library can reach the motion controller.

Program

```
#include <Newrick.h>

Newrick nr;

void setup() {
  Serial.begin(115200);
  delay(2000); // give the Serial Monitor time to attach

  nr.begin(); // starts I2C on SDA = 8, SCL = 9
  Serial.println("Newrick ready");

  if (nr.updateSensors()) {
    Serial.print("Battery: ");
    Serial.print(nr.batteryVolts, 2);
    Serial.println(" V");
    Serial.println("Motion controller responding.");
  } else {
    Serial.println("No reply from the motion controller.");
  }
}

void loop() {
  if (nr.updateSensors()) {
    Serial.print("Battery: ");
    Serial.print(nr.batteryVolts, 2);
    Serial.print(" V | Button: ");
    Serial.println(nr.buttonState);
  }
  delay(500);
}
```

Program Explanation

- `#include <Newrick.h>`
 - Includes the Newrick library, which provides the link to the motion controller.
- `Newrick nr;`
 - Creates the library object. Every Newrick program needs exactly one, and by convention it is named `nr`.
- `Serial.begin(115200);`
 - Initialises serial communication at **115200 baud**.
- `delay(2000);`
 - Waits **2 seconds** so that the USB serial port is enumerated before the first message is printed. Without it the opening lines are often lost.
- `nr.begin();`
 - Starts the I2C bus on the board's default pins and clears the library's stored readings. This must be called once in `setup()` before any other Newrick function.



ENVIRONMENT SETUP

- `nr.updateSensors();`
 - Asks the motion controller for a fresh set of readings and returns **true** if the reply arrived intact. Here it doubles as a test that the two controllers are talking.
- `nr.batteryVolts`
 - The pack voltage in volts, refreshed by the call above.
- `nr.buttonState`
 - The state of the onboard user button, also refreshed by the call above.

Working

- Connect the board to your computer with the USB-C cable and switch the battery on.
- Press **Upload**. The IDE compiles the sketch and writes it to the ESP32-S3.
- Open the Serial Monitor at **115200 baud**.
- **Neurick ready** appears, followed by a battery voltage that matches your pack and a button value that changes from 0 to a non-zero number while the user button is held down.
- If you see **No reply from the motion controller**, the I2C link is not working — check that the battery is switched on, since the motion controller is not powered from USB alone.